

Programming for AI

AI-2001

**Constructors, Variable Types,
Access Specifier**

National University of Computer & Emerging Sciences – NUCES – Karachi



National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2001

Programming for AI

Contents

1. What is a Constructor?	2
2. Types of Constructors in Python?	2
2.1 Default Constructor	2
2.2 Non-Parameterized Constructor	4
2.3 Parameterized Constructor	5
3. What is a Destructor?	6
3.1 How Destructors Works?	6
4. Variable Types in Context with Classes	7
4.1 Instance Variable	7
4.2 Instance Variable	7
5. Access Specifiers	8
5.1 Public Attributes	8
5.2 Protected Attributes	8
5.3 Private Attributes	9
5.3.1 Name Mangling	9

1. What is a Constructor?

When an object of a class is created in Python, a constructor is a special kind of method that is automatically called. Instance variables of a class are initialized using constructors. They are defined by using the `__init__()` method and share the same name as the class.

An important concept in object-oriented programming, constructors are crucial. They enable programmers to produce objects of a class with their attributes set to initial values. By employing constructors, we can guarantee that the object is created in a usable state and prevent any potential undefined behavior.

2. Types of Constructors in Python?

In Python, there are three different kinds of constructors:

2.1 Default Constructor

This is the simplest constructor type and does not accept any parameters. Python automatically creates a default constructor when a class does not have one defined explicitly. All of the **class**' instance variables are initialized to their default values by the default constructor.

```
class Student:
    # Default constructor
    def __init__(self):
        self.name = "Unknown"
        self.age = 0
        self.course = "Not Enrolled"
        self.grade = "No Grade"

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"Enrolled Course: {self.course}")
        print(f"Grade: {self.grade}")
        print("-----")

    def enroll(self, course_name):
        self.course = course_name
        print(f"{self.name} has been enrolled in {self.course}.")

    def assign_grade(self, grade):
        self.grade = grade
        print(f"{self.name} has been assigned grade {self.grade}.")

student1 = Student()
student2 = Student()
```

```

print("Before Enrollment:")
student1.display_details()
student2.display_details()

student1.name = "Ali"
student1.age = 20
student1.enroll("Introduction to Programming")

student2.name = "Rizwan"
student2.age = 22
student2.enroll("Database Systems")

student1.assign_grade("A")
student2.assign_grade("B+")

print("After Enrollment and Grading:")
student1.display_details()
student2.display_details()

```

Output

```

Before Enrollment:
Student Name: Unknown
Age: 0
Enrolled Course: Not Enrolled
Grade: No Grade
-----
Student Name: Unknown
Age: 0
Enrolled Course: Not Enrolled
Grade: No Grade
-----
Ali has been enrolled in Introduction to Programming.
Rizwan has been enrolled in Database Systems.
Ali has been assigned grade A.
Rizwan has been assigned grade B+.
After Enrollment and Grading:
Student Name: Ali
Age: 20
Enrolled Course: Introduction to Programming
Grade: A
-----
Student Name: Rizwan
Age: 22
Enrolled Course: Database Systems
Grade: B+

```

This program defines a **Student** class with a default constructor that initializes student attributes (**name**, **age**, **course**, **grade**) to default values. It includes methods to:

- **display_details:** Prints the student's information.
- **enroll:** Updates the course a student is enrolled in.
- **assign_grade:** Assigns a grade to the student.

The program creates two **student** objects, modifies their details by enrolling them in courses and assigning grades, and then displays their updated information.

2.2 Non-Parameterized Constructor

This kind of constructor is explicitly defined by the programmer but does not accept any parameters. It is also known as the constructor without arguments. The instance variable values are initialized to some predetermined default values in this kind of constructor.

```
class Car:
    # Non-parameterized constructor
    def __init__(self):
        self.make = "Unknown"
        self.model = "Unknown"
        self.year = 0
        self.color = "Unpainted"

    def display_details(self):
        print(f"Car Make: {self.make}")
        print(f"Model: {self.model}")
        print(f"Year: {self.year}")
        print(f"Color: {self.color}")
        print("-----")

    def update_details(self, make, model, year, color):
        self.make = make
        self.model = model
        self.year = year
        self.color = color

car1 = Car()
car2 = Car()

print("Before Update:")
car1.display_details()
car2.display_details()

car1.update_details("Toyota", "Corolla", 2020, "Red")
car2.update_details("Honda", "Civic", 2018, "Blue")

print("After Update:")
car1.display_details()
car2.display_details()
```

Output

```
Before Update:
Car Make: Unknown
Model: Unknown
```

```

Year: 0
Color: Unpainted
-----
Car Make: Unknown
Model: Unknown
Year: 0
Color: Unpainted
-----
After Update:
Car Make: Toyota
Model: Corolla
Year: 2020
Color: Red
-----
Car Make: Honda
Model: Civic
Year: 2018
Color: Blue
-----

```

- **Non-parameterized constructor (__init__):** It sets default values (**make, model, year, color**) when a **Car** object is created.
- **update_details** method: Allows updating the car's attributes.
- **display_details** method: Prints the car's details.

2.3 Parameterized Constructor

This type of constructor accepts one or more parameters and initializes the instance variables of the class with the values passed as arguments. It is also called the argument constructor.

```

class Car:
    # Parameterized constructor
    def __init__(self, make, model, year, color):
        self.make = make
        self.model = model
        self.year = year
        self.color = color

    def display_details(self):
        print(f"Car Make: {self.make}")
        print(f"Model: {self.model}")
        print(f"Year: {self.year}")
        print(f"Color: {self.color}")
        print("-----")

car1 = Car("Toyota", "Corolla", 2020, "Red")
car2 = Car("Honda", "Civic", 2018, "Blue")

print("Car Details:")
car1.display_details()
car2.display_details()

```

Output	Car Details: Car Make: Toyota Model: Corolla Year: 2020 Color: Red ----- Car Make: Honda Model: Civic Year: 2018 Color: Blue -----
--------	--

3. What is a Destructor?

Destructors are procedures that are automatically called when an object is destroyed or garbage-collected. The **del()** method is used to define the destructor method in Python.

3.1 How Destructors Works?

The constructor of a class is automatically invoked when an object of that class is created in order to initialize the class' instance variables. Similar to this, the destructor is automatically called when an object is destroyed or garbage collected to release the resources used by the object. When an object is no longer in use, Python automatically calls the destructor rather than the programmer explicitly doing so.

```
class Car:
    # Constructor
    def __init__(self, make, model):
        self.make = make
        self.model = model
        print(f"A new car {self.make} {self.model} has been created.")

    def __del__(self):
        print(f"The car {self.make} {self.model} is being destroyed.")

    def display_details(self):
        print(f"Car Make: {self.make}, Model: {self.model}")

car1 = Car("Toyota", "Corolla")

car1.display_details()

del car1

print("End of program.")
```

Output	A new car Toyota Corolla has been created. Car Make: Toyota, Model: Corolla The car Toyota Corolla is being destroyed. End of program.
--------	---

- **Constructor (__init__):** Initializes the **car** with a make and model when the object is created.
- **Destructor (__del__):** This method is automatically called when the object is about to be destroyed (either explicitly using del or when the object goes out of scope).
- When the **car1** object is deleted, the destructor is called, and the message in **__del__** is printed.

4. Variable Types in Context with Classes

In context to classes, variables can be classified into instance variables and class variables. Each type has a specific scope and behaviour within a program.

4.1 Instance Variable

Instance variables are variables that are unique to each object (instance) of a class. Each object has its own copy of instance variables.

- **Scope:** They are defined inside the **__init__** constructor or other methods with self.
- **Usage:** They are used to store information unique to each instance of the class.

```
class Student:
    def __init__(self, name, age):
        self.name = name # Instance variable
        self.age = age # Instance variable

student1 = Student("Ali", 20)
student2 = Student("Rizwan", 22)

print(student1.name, student1.age)
print(student2.name, student2.age)
```

Output	Ali 20 Rizwan 22
---------------	---------------------

4.2 Instance Variable

Class variables are shared across all objects of a class. Only one copy of a class variable is maintained, and it is shared among all instances.

- **Scope:** They are defined within the class but outside any method, and they are accessed using the class name or through objects.
- **Usage:** They are used when we want data to be shared among all instances of the class.

```
class Student:
```

<pre> # Class variable school = "FAST - NUCES" def __init__(self, name, age): self.name = name # Instance variable self.age = age # Instance variable student1 = Student("Alice", 20) student2 = Student("Bob", 22) print(student1.school) print(student2.school) print(Student.school) </pre>	
Output	FAST - NUCES FAST - NUCES FAST - NUCES

5. Access Specifiers

In Python, access specifiers (public, protected, and private) control the accessibility of variables and methods within a class.

5.1 Public Attributes

Public attributes are accessible from anywhere, both inside and outside the class.

Convention: There is no leading underscore (_).

Usage: Most attributes in Python are public by default.

<pre> class Car: def __init__(self, make, model): self.make = make # Public attribute self.model = model # Public attribute car = Car("Toyota", "Corolla") print(car.make) # Accessible print(car.model) # Accessible </pre>	
Output	Toyota Corolla

5.2 Protected Attributes

Protected attributes are intended to be accessible only within the class and its subclasses. However, in Python, they can still be accessed from outside, but by convention, they should be treated as non-public.

Convention: A single leading underscore (_).

Usage: While Python doesn't enforce protection, it is a soft convention that indicates to developers that the attribute is intended for internal use.

```
class Car:
    def __init__(self, make, model):
        self._make = make # Protected attribute
        self._model = model # Protected attribute

car = Car("Toyota", "Corolla")
print(car._make) # Accessible but discouraged by convention
```

Output	Toyota
---------------	--------

5.3 Private Attributes

Private attributes are intended to be accessible only within the class and are not directly accessible from outside the class. Python achieves this through name mangling.

Convention: Two leading underscores (`__`).

Usage: Private attributes are more strongly protected from outside access, though still not completely inaccessible.

```
class Car:
    def __init__(self, make, model):
        self.__make = make # Private attribute
        self.__model = model # Private attribute

    def get_make(self): # Public method to access private attribute
        return self.__make

car = Car("Toyota", "Corolla")
# print(car.__make) # This would raise an AttributeError
print(car.get_make()) # Accessing private attribute through a method
```

Output	Toyota
---------------	--------

5.3.1 Name Mangling

Private attributes are actually stored as `__ClassName__` attribute internally. So, they can be accessed like this (but it's discouraged):

```
print(car.__Car__make)
```